

KI-gestützte Software-Entwicklung: Agent-Architektur in der Praxis

Wie strukturierte KI-Teams Codequalität liefern, die über „es läuft halt“ hinausgeht

1. Das Problem: Warum naive KI-Nutzung enttäuscht

Die typische KI-Nutzung beim Coden: Chat öffnen, Anforderung beschreiben, Code zurückbekommen. Das Ergebnis läuft meistens — aber Struktur, Fehlerbehandlung und Konventionstreue sind bestenfalls zufällig.

Warum?

- **Kein Projektkontext:** Die KI kennt weder Codebase, noch Architektur, noch Konventionen
- **Kein Review-Prozess:** Der Output wird 1:1 übernommen oder manuell nachgebessert
- **Kein Quality Gate:** Kein Linting, keine Tests, kein Build-Check vor der Übernahme
- **Kein Spezialistenwissen:** Eine generische KI-Session weiß nicht, dass in diesem Projekt `.unwrap()` verboten ist

Das ist exakt das Szenario, in dem auch ein menschlicher Entwickler schlechte Arbeit abliefern würde: ohne Onboarding, ohne Review, ohne CI/CD. Nicht weil er unfähig ist, sondern weil der Prozess fehlt.

2. Die Lösung: Agent-Architektur

2.1 Die Bausteine

Claude Code (Anthropics CLI-Tool) unterstützt seit 2025 ein System, das über einfache Chat-Interaktion weit hinausgeht:

CLAUDE.md — Das Projektgedächtnis

Eine Markdown-Datei im Repo-Root, die bei jeder Claude-Session automatisch geladen wird. Enthält Projektstruktur, Architektur-Entscheidungen, Coding-Konventionen und harte Regeln. Vergleichbar mit einem Onboarding-Dokument — nur dass die KI es bei jeder Session liest und sich daran hält.

Agents — Spezialisierte Entwickler

Eigenständige Claude-Sessions mit:

- **Eigenem System Prompt** — Identität, Denkweise, Zuständigkeit

- **Eingeschränktem Toolset** — ein UI-Agent braucht kein Bash, ein Stratege kein Write
- **Eigener Modellwahl** — teure Modelle für Urteilsvermögen, günstige für mechanische Tasks
- **Begrenzter Turn-Anzahl** — verhindert endlose Schleifen

Entscheidend: Ein Agent bekommt nicht nur gesagt **was** er tun soll, sondern **wie er denken** soll. „Du bist ein erfahrener Rust-Entwickler, der Fehlerbehandlung ernst nimmt“ produziert fundamental anderen Code als ein generischer Prompt.

Skills — Automatisierte Workflows

Wiederverwendbare Prozeduren für mechanische Aufgaben: Linting, Tests, Builds, Commits, Releases. Skills brauchen kein Urteilsvermögen — sie führen definierte Schritte aus. Deshalb laufen sie auf dem günstigsten Modell.

Main Agent — Der Tech Lead

Der zentrale Orchestrator. Er plant, delegiert, **reviewt** und eskaliert. Er codet selbst nicht — wie ein Tech Lead im echten Team.

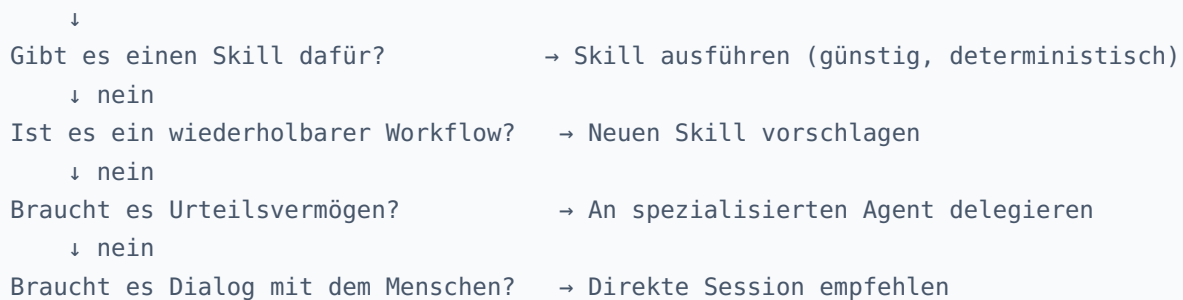
Plugins — Die Packaging-Schicht

Seit Oktober 2025 können Skills, Agents, Hooks, MCP-Server und LSP-Konfigurationen als **Plugin** gebündelt werden. Ein einziger Install-Befehl gibt dem gesamten Team identisches Tooling — keine manuelle Einrichtung, kein „Works on my machine“. Über 9.000 Extensions existieren bereits im Ökosystem.

Für Teams entscheidend: Eine komplette Entwicklungs-Pipeline (Agents, Skills, Quality Gates) lässt sich als Plugin paketieren und per `claude plugin install` verteilen. Neue Teammitglieder sind sofort produktiv — mit denselben Prozessen, Konventionen und Checks.

2.2 Die Entscheidungslogik

Aufgabe kommt rein



Diese Hierarchie optimiert gleichzeitig Qualität und Kosten. Mechanische Arbeit wird mechanisch erledigt. Urteilsvermögen wird dort eingesetzt, wo es gebraucht wird.

3. Die Pipeline am konkreten Beispiel

Das folgende Beispiel stammt aus einem produktiven Projekt: eine Cross-Platform Sprache-zu-Text-App (Rust/Tauri Backend, React Frontend, Kotlin Android). ~15.000 LOC, echte Nutzer, regelmäßige Releases.

Team-Aufstellung

ROLLE	ZUSTÄNDIGKEIT	MODELL	WARUM?
Tech Lead	Planung, Delegation, Review	Opus	Strategisches Urteilsvermögen
rust-core	Audio, STT-Pipeline, LLM-Cleanup, Hotkeys	Sonnet	Inhaltliche Entwicklung
ui-dev	React-Overlay, Settings-UI, Styling	Sonnet	Inhaltliche Entwicklung
android-platform	Kotlin: Floating Bubble, Permissions	Sonnet	Inhaltliche Entwicklung
product-strategist	Positionierung, Pricing, Roadmap	Sonnet	Recherche + Urteil

Stufe 1: Planung

Der Entwickler sagt: „Wir brauchen einen Offline-Fallback für die Spracherkennung.“

Der Tech Lead:

1. Liest die Architektur-Dokumentation
2. Zerlegt das Feature in Tasks mit Agent-Zuordnung
3. Identifiziert Abhängigkeiten zwischen Tasks
4. Persistiert den Plan als Briefing-Datei

Warum besser als ein Prompt: Die Zerlegung passiert vor dem Coden. Jeder Agent bekommt nur seinen Teil mit klarem Scope. Abhängigkeiten sind explizit.

Stufe 2: Implementierung + Review

Der Tech Lead delegiert an den passenden Sub-Agent. Dieser kennt durch System Prompt und CLAUDE.md die bestehende Architektur, verbotene Patterns und Konventionen.

```

Tech Lead → beauftragt Sub-Agent
↓
Sub-Agent → implementiert (kennt Architektur + Konventionen)
↓
Tech Lead → reviewt:
  ✓ Architektur-konform?
  ✓ Fehlerbehandlung sauber?
  ✓ Tests vorhanden?
  x Problem? → Korrektur-Anweisung
↓
Sub-Agent → überarbeitet (max. 2 Runden)
↓
✓ Approved → weiter zu Stufe 3
x 2× gescheitert → Abbruch, Mensch wird informiert

```

Warum besser als ein Prompt: Ein zweites Modell reviewt — wie bei menschlichen Code-Reviews fängt ein zweites Augenpaar Fehler. Die 2-Runden-Grenze verhindert endloses Verschlimmbessern.

Stufe 3: Qualitätssicherung

QA	/lint-fix <code>cargo fmt</code> + <code>clippy</code> (600+ Regeln) + <code>prettier</code> + <code>eslint</code> — alles mit Auto-Fix
QA	/build Kompiliert für Windows UND Android — fängt Typ-Fehler, fehlende Imports, ABI-Probleme
QA	/run-tests <code>cargo test</code> + <code>npm test</code> — kein Code ohne grüne Tests im Commit
!	/debug-error Nur bei Fehlschlag: Analyse im Codebase-Kontext, gezielter Fix statt Raten

Warum besser als ein Prompt: Diese Checks passieren automatisch als Pipeline-Stufe, nicht „wenn der Entwickler dran denkt“. Ein Linter hat keine schlechten Tage.

Stufe 4: Integration

Conventional Commits (`feat:` , `fix:` , `refactor:`). Nur relevante Dateien werden gestaged. Architektur-Entscheidungen werden in Knowledge-Dateien festgehalten — nicht nur im Chat.

Stufe 5: Release

Ein einziger Befehl (`/release`) führt aus:

1. Version-Bump in drei Konfigurationsdateien
2. Windows-Build (inkl. Installer + Update-Signatur)
3. Android-Build (APK)
4. Auto-Update-Manifest generieren
5. Git Commit + Push → Sync zum Public Repo → GitHub Release

Meta-Qualität: Der Skill Creator

Die Pipeline sichert die Qualität des Codes. Aber wer sichert die Qualität der Arbeitsanweisungen (Skills) an die KI? Seit März 2026 gibt es dafür den **Skill Creator** — ein offizielles Anthropic-Plugin, das Software-Engineering-Methoden auf die Skill-Entwicklung anwendet:

MODUS	FUNKTION
Create	Skill aus natürlicher Sprache erstellen
Eval	Tests definieren, die prüfen ob ein Skill tut was er soll — parallel, in isolierten Kontexten
Improve	Skill automatisch optimieren: Train/Test-Split (60/40), bis zu 5 Iterationen, Anti-Overfitting
Benchmark	Standardisierte Messung: Pass-Rate, Token-Verbrauch, Laufzeit — vergleichbar über Versionen

Vier spezialisierte Sub-Agents (Executor, Grader, Comparator, Analyzer) arbeiten parallel und isoliert. Das Ergebnis: Skill-Qualität wird nicht „gehofft“, sondern **gemessen, verglichen und iterativ verbessert**.

Nicht nur der Code wird getestet — auch die Prozesse, die den Code erzeugen, durchlaufen Evals und Benchmarks. Qualitätssicherung auf zwei Ebenen.

4. Häufige Einwände — und wie die Architektur sie adressiert

„Der Code läuft, aber ist schlecht strukturiert“

Architektur-Kontext löst das. CLAUDE.md definiert Projektstruktur und Entscheidungen. Agents kennen die bestehende Codebase und schreiben passenden Code, nicht irgendeinen. Der Main Agent reviewt auf Architektur-Konformität.

„Fehlerbehandlung ist oberflächlich“

Harte Regeln im Kontext. CLAUDE.md kann Regeln wie „kein `.unwrap()` in Nicht-Test-Code“ definieren. Der Linter fängt Verstöße automatisch. Der Review prüft gezielt Fehlerbehandlung.

„Es fehlen Tests“

Tests als Pipeline-Stufe. `/run-tests` ist Teil der Pipeline — Code ohne grüne Tests erreicht keinen Commit. Agents können angewiesen werden, Tests als Teil der Implementierung zu schreiben.

„Ich verliere die Kontrolle über meinen Code“

Der Mensch definiert die Regeln. Er schreibt CLAUDE.md, System Prompts und Skill-Definitionen. Er definiert wie gearbeitet wird. Jeder Agent hat begrenzte Tools und begrenzte Autonomie. Nach 2 gescheiterten Versuchen wird an den Menschen eskaliert. Nichts wird gepusht ohne explizite Freigabe.

„Ein Modell reviewt sich quasi selbst“

Strukturelle Trennung auf drei Ebenen. (1) Implementierung und Review laufen in getrennten Sessions mit getrenntem Kontext — verschiedene Modelle möglich (Sonnet implementiert, Opus reviewt). (2) Linter, Tests und Build-Checks sind komplett modellunabhängig. (3) Seit 2026 werden selbst die Skill-Definitionen mit formalen Evals und Benchmarks geprüft (Skill Creator) — Qualitätssicherung auf der Meta-Ebene.

5. Modellwahl als Kostenhebel

Nicht jede Aufgabe braucht das teuerste Modell:

MODELL	STÄRKE	EINSATZ	RELATIVER PREIS
Opus	Strategisches Denken, Review	Main Agent, Architektur	€€€
Sonnet	Solide Implementierung	Sub-Agents, inhaltliche Arbeit	€€
Haiku	Schnelle Ausführung	Skills (Lint, Test, Build, Commit)	€

Ein naiver Ansatz schickt alles an das teuerste Modell. Die Agent-Architektur setzt Opus nur dort ein, wo Urteilsvermögen gebraucht wird — und spart bei mechanischen Tasks über 90% der Kosten.

6. Was das für ein Entwicklungsteam bedeutet

VORHER	NACHHER
Entwickler schreibt Code	Entwickler definiert wie Code geschrieben wird
KI als Autovervollständigung	KI als strukturiertes Entwicklungsteam
Qualität hängt vom Prompt ab	Qualität hängt von der Architektur ab
Ein Modell, ein Chat, ein Versuch	Spezialisierte Agents mit Review-Loops
„Es läuft halt“	Linting, Tests, Build-Checks als Pipeline
Jeder richtet sich sein Setup ein	Plugin installieren — ganzes Team hat identische Prozesse

Die Qualität von KI-generiertem Code ist keine Eigenschaft des Modells
—
sie ist eine Funktion der Architektur, in der das Modell arbeitet.

Ein erfahrener Entwickler ohne Code-Review, ohne CI/CD und ohne Architektur-Vorgaben produziert auch mittelmäßigen Code. Nicht weil er unfähig ist, sondern weil der Prozess fehlt. Dasselbe gilt für KI — nur dass die Architektur einmal aufgesetzt wird und danach konsistent arbeitet: ohne schlechte Tage, ohne vergessene Reviews, ohne „ich push das schnell ohne Tests“.

Basierend auf einem produktiven Projekt: Cross-Platform Desktop/Mobile-App, Rust/React/Kotlin, ~15.000 LOC. Regelmäßige Releases an echte Nutzer. Gesamte Entwicklung über Agent-Architektur mit Claude Code.